

Implementation of Simple Timer IP

Introduction

A timer IP (Intellectual Property) is a fundamental digital design component used extensively in microprocessors, SoCs (System-on-Chip), and embedded systems. This programmable countdown timer provides precise timing control for various applications including real-time systems, communication protocols, and power management units.

In this experiment, you will learn to design, implement, and verify a simple timer IP that:

- Starts from a preset value
- Counts down on each clock cycle
- Signals when it reaches zero
- Provides control through start, preset value, and done signals

The timer will be implemented in Verilog HDL and verified using Verilator simulation with GTKWave waveform analysis.

Applications

Timer IPs are used in:

- Microcontrollers and microprocessors
- Real-time operating systems (RTOS)
- Communication protocol stacks
- Power management circuits
- Watchdog timer implementations
- Pulse width modulation (PWM) generators
- Any system requiring precise timing control

By implementing a parameterized countdown timer, you will understand fundamental concepts of sequential digital design and IP development methodology.

Block Diagram

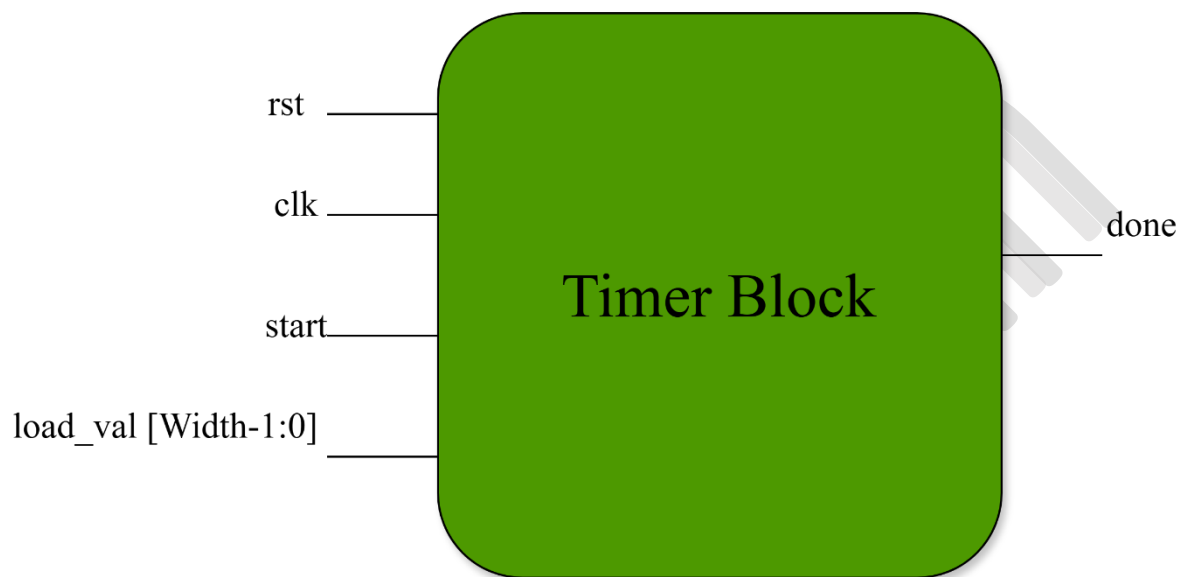


Fig: Block Diagram Timer IP

Procedure To Create Project

Step 1: Set up Project Directory

1.1: Create a directory for your lab:

```
mkdir timer_ip_lab
```

1.2: Open lab directory:

```
cd timer_ip_lab
```

Step 2: Create Design Files

2.1: Simple Timer IP Design (simple_timer.v)

Create a file for design : simple_timer.v

```
// A basic programmable countdown timer
```

```
`timescale 1ns/1ps

module simple_timer #(
    parameter WIDTH = 8    // Width of timer counter
) (
    input wire             clk,        // System clock
    input wire             rst,        // Synchronous reset
    input wire             start,      // Start the timer
    input wire [WIDTH-1:0] load_val,   // Initial countdown
    value
    output reg             done        // High when timer
    reaches zero
);

    reg [WIDTH-1:0] count; // Timer counter register
    reg             running; // Indicates timer is active
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count    <= 0;
            running <= 0;
            done     <= 0;
        end else begin
            if (start) begin
                // Load the preset value and start counting
                count    <= load_val;
                running <= 1;
                done     <= 0;
            end else if (running) begin
                if (count > 0) begin
```

```
        count <= count - 1;    // Decrement counter
    end else begin
        running <= 0;          // Stop timer
        done    <= 1;          // Raise done signal
    end
end
end
end
endmodule
```

Step 3: Create Testbench File

3.1: Create a file for testbench : simple_timer_tb.v

```
`timescale 1ns/1ps
module simple_timer_tb;
    reg clk = 0;
    reg rst = 1;
    reg start = 0;
    reg [7:0] load_val = 0;
    wire done;
    // Instantiate Timer DUT
    simple_timer #(.WIDTH(8)) uut (
        .clk(clk),
        .rst(rst),
        .start(start),
        .load_val(load_val),
        .done(done)
    );
endmodule
```

```
// Clock generation
always #5 clk = ~clk;

initial begin

    $dumpfile("simple_timer.vcd");
    $dumpvars(0, simple_timer_tb);

    // Reset
    #10 rst = 0;

    // Start timer with value 5
    #5 load_val = 8'd5; start = 1;
    #10 start = 0;

    // Wait for timer to finish
    #100;

    // Start timer again with value 3
    #5 load_val = 8'd3; start = 1;
    #10 start = 0;

    #50 $finish;

end

endmodule
```

Step 4: Simulation Script

4.1 Create a script file run.sh

```
#!/bin/bash
```

Step 1: Run Verilator to generate simulation files

```
verilator --binary -j 0 -Wall simple_timer.v simple_timer_tb.v \
--top-module simple_timer_tb --timing \
--CFLAGS "-std=c++20 -fcoroutines" --trace
```

Step 2: Check if obj_dir is created and move into it

```
cd obj_dir || { echo "obj_dir not found"; exit 1; }
```

Step 3: Compile the testbench using make

```
make -f Vsimple_timer_tb.mk Vsimple_timer_tb || { echo  
"Compilation failed"; exit 1; }
```

Step 4: Run the testbench simulation

```
./Vsimple_timer_tb || { echo "Simulation failed"; exit 1; }
```

Step 5: View the waveform using GTKWave

```
gtkwave simple_timer.vcd
```

4.2 : Save the script file.

4.3: Make the script executable:

```
chmod +x run.sh
```

4.4: Execute the simulation script:

```
./run.sh
```

Simulation Output:

The simulation demonstrates several key timer behaviors:

Key Observations:

- **count** decrements from `load_val` to 0 over successive clock cycles
- **running** signal indicates active countdown state
- **done** pulse occurs when timer reaches zero
- Enhanced timer responds to **enable** control for pause/resume functionality

Step 5: Viewing the Waveform

Signal Analysis in GTKWave:

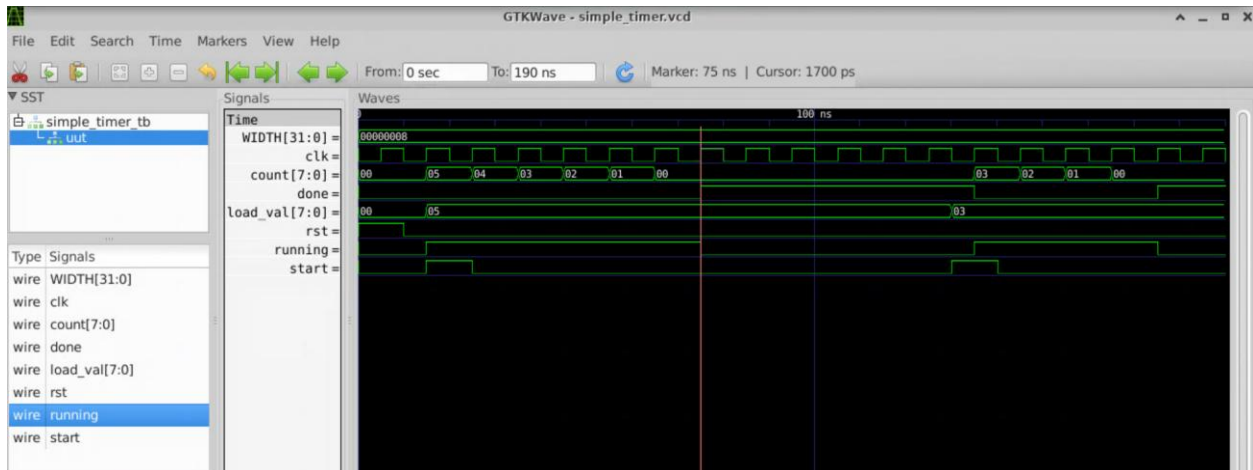
- **clk**: Continuous clock signal driving all operations
- **rst**: Reset pulse initializing all registers
- **start**: Trigger pulse to begin countdown operation
- **load_val**: Preset value loaded into timer
- **count**: Current timer value showing countdown progression
- **done**: Single-cycle pulse indicating timer completion
- **running**: Status signal showing active timer state

Power and Performance Observations:

- Timer only consumes dynamic power during active countdown
- Enable control allows power-efficient pause functionality

- Parameterized design supports different bit widths for various applications

Output Waveform Analysis:



- X-axis: Simulation time in nanoseconds.
- Y-axis: Signal logic levels (0 = low, 1 = high, numerical values shown in brackets)
- Clear correlation between countdown progression and control signal behavior

Design Extensions and Improvements:

The basic timer design can be enhanced with additional features:

Advanced Features:

- **Auto-reload capability:** Continuous operation mode
- **Interrupt generation:** CPU notification on timer completion
- **Prescaler support:** Slower counting for longer intervals
- **Up/down counting modes:** Bidirectional timer operation
- **Compare registers:** Multiple threshold detection
- **Cascade support:** Linking multiple timers for extended range

Applications in System Design:

- **Real-time task scheduling:** RTOS tick generation
- **Communication timeouts:** Protocol timeout detection
- **Watchdog functionality:** System health monitoring
- **PWM generation:** Motor control and power regulation
- **Frequency division:** Clock generation for slower peripherals

Conclusion:

This timer IP laboratory provides comprehensive experience in digital IP design methodology. Students learn fundamental concepts including:

- **Sequential logic design** using parameterized modules
- **State machine implementation** for timer control
- **Interface design** with standard control and status signals
- **Testbench development** for thorough verification
- **Waveform analysis** for debugging and validation

The experiment demonstrates how simple digital building blocks become essential IP components in complex system designs. Understanding timer implementation principles prepares students for advanced topics in embedded systems, real-time computing, and SoC design.

The comparative analysis between basic and enhanced timer variants illustrates design trade-offs between simplicity and functionality, teaching valuable lessons about IP development and system integration requirements.

This hands-on experience with timer IP design provides essential foundation knowledge for modern digital system development, where timing control remains a critical requirement across all application domains.